3 Domain models and metadata

This chapter covers

- Introducing the CaveatEmptor example application
- Implementing the domain model
- Examining object/relational mapping metadata options

The "Hello World" example in the previous chapter introduced you to Hibernate, Spring Data, and JPA, but it isn't useful for understanding the requirements of real-world applications with complex data models. For the rest of the book, we'll use a much more sophisticated example application—CaveatEmptor, an online auction system—to demonstrate JPA, Hibernate, and later Spring Data. (*Caveat emptor* means "Let the buyer beware.")

Major new features in JPA 2

A JPA persistence provider now integrates automatically with a Bean Validation provider. When data is stored, the provider automatically validates constraints on persistent classes.

The `Metamodel` API has also been added. You can obtain the names, properties, and mapping metadata of the classes in a persistence unit.

We'll start our discussion of the CaveatEmptor application by introducing its layered application architecture. Then you'll learn how to identify the business entities of a problem domain. You'll create a conceptual model of these entities and their attributes, called a *domain model*, and you'll implement it in Java by creating persistent classes. We'll spend some time exploring exactly what these Java classes should look like and where they fit within a typical layered application architecture. We'll also look at the persistence capabilities of the classes and at how this influences the application's design and implementation. We'll add Bean Validation, which will help you to automatically verify the integrity of the domain model data—both the persistent information and the business logic.

We'll then explore some mapping metadata options—the ways you tell Hibernate how the persistent classes and their properties relate to database tables and columns. This can be as simple as adding annotations directly in the Java source code of the classes or writing XML documents

that you eventually deploy along with the compiled Java classes that Hibernate accesses at runtime.

After reading this chapter, you'll know how to design the persistent parts of your domain model in complex real-world projects and what mapping metadata option you'll primarily prefer to use. Let's start with the example application.

3.1 The example CaveatEmptor application

The CaveatEmptor example is an online auction application that demonstrates ORM techniques, JPA, Hibernate, and Spring Data functionality. We won't pay much attention to the user interface in this book (it could be web-based or a rich client); we'll concentrate instead on the data access code.

To understand the design challenges involved in ORM, let's pretend the CaveatEmptor application doesn't yet exist and that we're building it from scratch. Let's start by looking at the architecture.

3.1.1 A layered architecture

With any nontrivial application, it usually makes sense to organize classes by concern. Persistence is one concern; others include presentation, workflow, and business logic. A typical object-oriented architecture includes layers of code that represent these concerns.

Cross-cutting concerns

There are also so-called *cross-cutting concerns*, which may be implemented generically, such as by framework code. Typical cross-cutting concerns include logging, authorization, and transaction demarcation.

A layered architecture defines interfaces between the code that implements the various concerns, allowing changes to be made to the way one concern is implemented without significant disruption to code in the other layers. Layering determines the kinds of inter-layer dependencies that occur. The rules are as follows:

- Layers communicate from top to bottom. A layer is dependent only on the interface of the layer directly below it.
- Each layer is unaware of any other layers except for the layer just below it and eventually of the layer above if it receives explicit requests from it.

Different systems group concerns differently, so they define different layers. The typical, proven, high-level application architecture uses three

layers: one each for presentation, business logic, and persistence, as shown in figure 3.1.

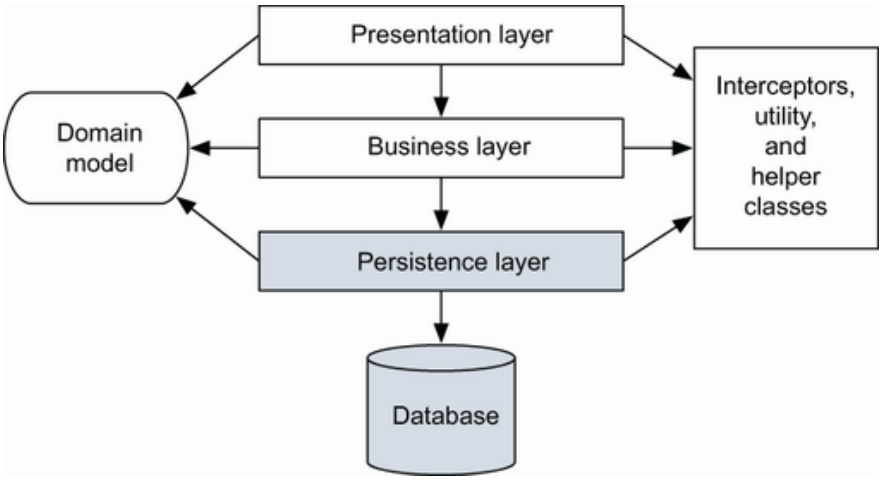

Figure 3.1 A persistence layer is the basis of a layered architecture.

- *Presentation layer*—The user interface logic is topmost. Code responsible for the presentation and control of page and screen navigation is in the presentation layer. The user interface code may directly access business entities of the shared domain model and render them on the screen, along with controls to execute actions. In some architectures, business entity instances might not be directly accessible by user interface code, such as when the presentation layer isn't running on the same machine as the rest of the system. In such cases, the presentation layer may require its own special data-transfer model, representing only a transmittable subset of the domain model. A good example of the presentation layer is working with a browser to interact with an application.
- *Business layer*—The business layer is generally responsible for implementing any business rules or system requirements that are part of the problem domain. This layer usually includes some kind of controlling component—code that knows when to invoke which business rule. In some systems, this layer has its own internal representation of the business domain entities. Alternatively, it may rely on a domain model implementation that's shared with the other layers of the application. A good example of the business layer is the code responsible for executing the business logic.
- *Persistence layer*—The persistence layer is a group of classes and components responsible for storing data to, and retrieving it from, one or more data stores. This layer needs a model of the business domain entities for which you'd like to keep a persistent state. The persistence layer is where the bulk of JPA, Hibernate, and Spring Data use takes place.

- *Database*—The database is usually external. It's the actual persistent representation of the system state. If an SQL database is used, the database includes a schema and possibly stored procedures for the execution of business logic close to the data. The database is the place where data is persisted for the long term.
- *Helper and utility classes*—Every application has a set of infrastructural helper or utility classes that are used in every layer of the application. These may include general-purpose classes or cross-cutting concern classes (for logging, security, and caching). These shared infrastructural elements don't form a layer because they don't obey the rules for inter-layer dependency in a layered architecture.

Now that we have a high-level architecture, we can focus on the business problem.

3.1.2 Analyzing the business domain

At this stage, you, with the help of domain experts, should analyze the business problems your software system needs to solve, identifying the relevant main entities and their interactions. The main goal behind the analysis and design of a domain model is to capture the essence of the business information for the application's purpose.

Entities are usually notions understood by users of the system: payment, customer, order, item, bid, and so forth. Some entities may be abstractions of less concrete things the user thinks about, such as a pricing algorithm, but even these are usually understandable to the user. You can find all these entities in the conceptual view of the business, sometimes called an *information model*.

From this business model, engineers and architects of object-oriented software create an object-oriented model, still at the conceptual level (no Java code). This model may be as simple as a mental image that exists only in the mind of the developer, or it may be as elaborate as a UML class diagram. Figure 3.2 shows a simple model expressed in UML.

Figure 3.2 A class diagram of a typical online auction model

This model contains entities that you're bound to find in any typical e-commerce system: category, item, and user. This domain model represents all the entities and their relationships (and perhaps their attributes). This kind of object-oriented model of entities from the problem domain, encompassing only those entities that are of interest to the user, is called a *domain model*. It's an abstract view of the real world.

Instead of using an object-oriented model, engineers and architects may start the application design with a data model. This can be expressed with an entity-relationship diagram, and it will contain the `CATEGORY`, `ITEM`, and `USER` entities, together with the relationships between them. We usually say that, concerning persistence, there is little difference between the two types of models; they're merely different starting points. In the end, which modeling language you use is secondary; we're most interested in the structure of and relationships between the business entities. We care about the rules that have to be applied to guarantee the integrity of the data (for example, the multiplicity of relationships included in the model) and the code procedures used to manipulate the data (usually not included in the model).

In the next section we'll complete our analysis of the CaveatEmptor problem domain. The resulting domain model will be the central theme of this book.

3.1.3 The CaveatEmptor domain model

The CaveatEmptor site will allow users to auction many different kinds of items, from electronic equipment to airline tickets. Auctions proceed according to the English auction strategy: users continue to place bids on an item until the bid period for that item expires, and the highest bidder wins.

In any store, goods are categorized by type and grouped with similar goods into sections and onto shelves. The auction catalog requires some kind of hierarchy of item categories so that a buyer can browse the categories or arbitrarily search by category and item attributes. Lists of items will appear in the category browser and search result screens. Selecting an item from a list will take the buyer to an item-detail view where an item may have images attached to it.

An auction consists of a sequence of bids, and one is the winning bid. User details will include name, address, and billing information.

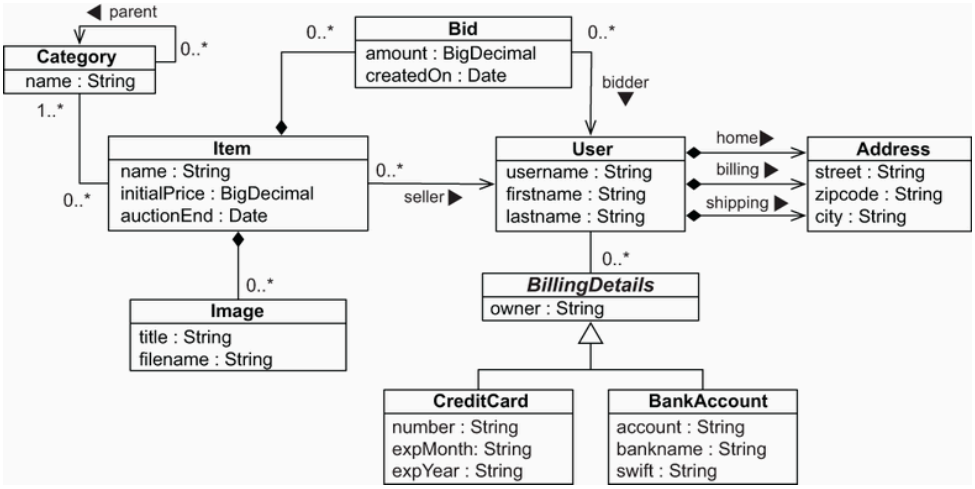

Figure 3.3 Persistent classes of the CaveatEmptor domain model and their relationships

The result of this analysis, the high-level overview of the domain model, is shown in figure 3.3. Let's briefly discuss some interesting features of this model:

- Each item can be auctioned only once, so you don't need to make `Item` distinct from any auction entities. Instead, you have a single auction item entity named `Item`. Thus, `Bid` is associated directly with `Item`. You model the `Address` information of a `User` as a separate class—a `User` may have three addresses for home, billing, and shipping. You allow the user to have many `BillingDetails`. Subclasses of an abstract class represent the various billing strategies (allowing for future extension).
- The application may nest a `Category` inside another `Category`, and so on. A recursive association, from the `Category` entity to itself, expresses this relationship. Note that a single `Category` may have multiple child categories but at most one parent. Each `Item` belongs to at least one `Category`.
- This representation isn't the *complete* domain model; it's only the classes for which you need persistence capabilities. You'll want to store and load instances of `Category`, `Item`, `User`, and so on. We have simplified this high-level overview a little; we'll make modifications to these classes when needed for more complex examples.
- The entities in a domain model should encapsulate state and behavior. For example, the `User` entity should define the name and address of a customer *and* the logic required to calculate the shipping costs for items (to this particular customer).

- There might be other classes in the domain model that only have transient runtime instances. Consider a `WinningBidStrategy` class encapsulating the fact that the highest bidder wins an auction. This might be called by the business layer (controller) code when checking the state of an auction. At some point you might have to figure out how the tax should be calculated for sold items or how the system should approve a new user account. We don't consider such business rules or domain model behavior to be unimportant; rather, those concerns are mostly orthogonal to the problem of persistence.

Now that you have a (rudimentary) application design with a domain model, the next step is to implement it in Java.

ORM without a domain model

Object persistence with full ORM is most suitable for applications based on a rich domain model. If your application doesn't implement complex business rules or complex interactions between entities, or if you have few entities, you may not need a domain model. Many simple and some not-so-simple problems are perfectly suited to table-oriented solutions, where the application is designed around the database data model instead of around an object-oriented domain model and the logic is often executed in the database (with stored procedures).

It's also worth considering the learning curve: once you're proficient with Hibernate and Spring Data, you'll use them for all applications—even something as a simple SQL query generator and result mapper. If you're just learning ORM, a trivial use case may not justify the time and overhead involved.

3.2 Implementing the domain model

Let's start with an issue that any implementation must deal with: the separation of concerns—which layer is concerned with what responsibility. The domain model implementation is usually a central, organizing component; it's reused heavily whenever you implement new application functionality. For this reason, you should go to some lengths to ensure that non-business concerns don't leak into the domain model implementation.

3.2.1 Addressing leakage of concerns

When concerns such as persistence, transaction management, or authorization start to appear in the domain model classes, this is an example of leakage of concerns. The domain model implementation is important code that shouldn't depend on orthogonal APIs. For example, code in the domain model shouldn't call the database directly or through an interme-

diate abstraction. This will allow you to reuse the domain model classes virtually anywhere.

The architecture of the application includes the following layers:

- The presentation layer can access instances and attributes of domain model entities when rendering views. The user may use the front end (such as a browser) to interact with the application. This concern should be separate from the concerns of the other layers.
- The controller components in the business layer can access the state of domain model entities and call methods of these entities. This is where the business calculations and logic are executed. This concern should be separate from the concerns of the other layers.
- The persistence layer can load instances of domain model entities from and store them to the database, preserving their state. This is where the information is persisted for a long time. This concern should also be separate from the concerns of the other layers.

Preventing the leakage of concerns makes it easy to unit test the domain model without the need for a particular runtime environment or container or for mocking any service dependencies. You can write unit tests that verify the correct behavior of your domain model classes without any special test harness. (Here we're talking about unit tests such as "calculate the shipping cost and tax," not performance and integration tests such as "load from the database" and "store in the database.")

The Jakarta EE standard solves the problem of leaky concerns with metadata such as annotations within your code or external XML descriptors. This approach allows the runtime container to implement some predefined cross-cutting concerns—security, concurrency, persistence, transactions, and remoteness—in a generic way by intercepting calls to application components.

JPA defines the *entity class* as the primary programming artifact. This programming model enables transparent persistence, and a JPA provider such as Hibernate also offers automated persistence. Hibernate isn't a Jakarta EE runtime environment, and it's not an application server. It's an implementation of the ORM technique.

3.2.2 Transparent and automated persistence

We use the term *transparent* to refer to a complete separation of concerns between the persistent classes of the domain model and the persistence layer. The persistent classes are unaware of—and have no dependency on—the persistence mechanism. From inside the persistent classes, there is no reference to the outside persistence mechanism. We use the term *automatic* to refer to a persistence solution (your annotated domain, the layer,

and the mechanism) that relieves you of handling low-level mechanical details, such as writing most SQL statements and working with the JDBC API. As a real-world use case, let's analyze how transparent and automated persistence is reflected at the level of the `Item` class.

The `Item` class of the CaveatEmptor domain model shouldn't have any runtime dependency on any Jakarta Persistence or Hibernate API. Furthermore, JPA doesn't require that any special superclasses or interfaces be inherited or implemented by persistent classes. Nor are any special classes used to implement attributes and associations. You can reuse persistent classes outside the context of persistence, such as in unit tests or in the presentation layer. You can create instances in any runtime environment with the regular Java `new` operator, preserving testability and reusability.

In a system with transparent persistence, instances of entities aren't aware of the underlying data store; they need not even be aware that they're being persisted or retrieved. JPA externalizes persistence concerns to a generic persistence manager API. Hence, most of your code, and certainly your complex business logic, doesn't have to concern itself with the current state of a domain model entity instance in a single thread of execution. We regard transparency as a requirement because it makes an application easier to build and maintain. Transparent persistence should be one of the primary goals of any ORM solution.

Clearly, no automated persistence solution is completely transparent: every automated persistence layer, including JPA and Hibernate, imposes some requirements on the persistent classes. For example, JPA requires that collection-valued attributes be typed to an interface such as `java.util.Set` or `java.util.List` and not to an actual implementation such as `java.util.HashSet` (this is good practice anyway). Similarly, a JPA entity class has to have a special attribute, called the *database identifier* (which is also less of a restriction but is usually convenient).

You now know that the persistence mechanism should have minimal effect on how you implement a domain model and that transparent and automated persistence is required. Our preferred programming model for achieving this is POJO.

NOTE POJO is the acronym for *Plain Old Java Objects*. Martin Fowler, Rebecca Parsons, and Josh Mackenzie coined this term in 2000.

At the beginning of the 2000s, many developers started talking about POJO, a back-to-basics approach that essentially revives JavaBeans, a component model for UI development, and reapplies it to the other layers of a system. Several revisions of the EJB and JPA specifications brought us new lightweight entities, and it would be appropriate to call them *persis-*

tence-capable JavaBeans. Java engineers often use all these terms as synonyms for the same basic design approach.

You shouldn't be too concerned about which terms we use in this book; our ultimate goal is to apply the persistence aspect as transparently as possible to Java classes. Almost any Java class can be persistence-capable if you follow some simple practices. Let's see what this looks like in code.

NOTE To be able to execute the examples from the chapter's source code, you'll need first to run the Ch03.sql script. The examples use a MySQL server with the default credentials: a username of *root* and no password.

3.2.3 Writing persistence-capable classes

Supporting fine-grained and rich domain models is a major Hibernate objective. This is one reason we work with POJOs. In general, using fine-grained objects means having more classes than tables.

A persistence-capable plain old Java class declares attributes, which represent state, and business methods, which define behavior. Some attributes represent associations to other persistence-capable classes.

The following listing shows a POJO implementation of the `User` entity of the domain model (example 1 in the `domainmodel` folder in the source code). Let's walk through this code.

Listing 3.1 POJO implementation of the `User` class

```
Path: Ch03/domainmodel/src/main/java/com/manning/javapersistence/ch03/e
➥ /User.java

\1 User {

    private String username;

    public String getUsername() {
        return username;
    }

    public void setUsername(String username) {
        this.username = username;
    }

}
```

The class can be abstract and, if needed, extend a non-persistent class or implement an interface. It must be a top-level class, not be nested within

another class. The persistence-capable class and any of its methods *shouldn't* be final (this is a requirement of the JPA specification). Hibernate is not so strict, and it will allow you to declare final classes as entities or as entities with final methods that access persistent fields. However, this is not a good practice, as this will prevent Hibernate from using the proxy pattern for performance improvement. In general, you should follow the JPA requirements if you would like your application to remain portable between different JPA providers.

Hibernate and JPA require a constructor with no arguments for every persistent class. Alternatively, if you do not write a constructor at all, Hibernate will use the default Java constructor. Hibernate calls classes using the Java Reflection API on such no-argument constructors to create instances. The constructor need not be public, but it has to be at least package-visible for Hibernate to use runtime-generated proxies for performance optimization.

The properties of the POJO implement the attributes of the business entities, such as the `username` of `User`. You'll usually implement properties as private or protected member fields, together with public or protected property accessor methods: for each field you'll need a method for retrieving its value and another for setting its value. These methods are known as the *getter* and *setter*, respectively. The example POJO in listing 3.1 declares getter and setter methods for the `username` property.

The JavaBean specification defines the guidelines for naming accessor methods; this allows generic tools like Hibernate to easily discover and manipulate property values. A getter method name begins with `get`, followed by the name of the property (with the first letter in uppercase). A setter method name begins with `set` and similarly is followed by the name of the property. You may begin getter method names for Boolean properties with `is` instead of `get`.

Hibernate doesn't require accessor methods. You can choose how the state of an instance of your persistent classes should be persisted. Hibernate will either directly access fields or call accessor methods. Your class design isn't disturbed much by these considerations. You can make some accessor methods non-public or completely remove them and then configure Hibernate to rely on field access for these properties.

Making property fields and accessor methods private, protected, or package visible

Typically you will not allow direct access to the internal state of your class, so you won't make attribute fields public. If you make fields or methods private, you're effectively declaring that nobody should ever ac-

cess them; only you are allowed to do that (or a service like Hibernate). This is a definitive statement.

There are often good reasons for someone to access your "private" internals—usually to fix one of your bugs—and you only make people angry if they have to fall back to reflection access in an emergency. Instead, you might assume or know that the engineer who comes after you has access to your code and knows what they're doing.

Although trivial accessor methods are common, one of the reasons we like to use JavaBeans-style accessor methods is that they provide encapsulation: you can change the hidden internal implementation of an attribute without making any changes to the public interface. If you configure Hibernate to access attributes through methods, you abstract the internal data structure of the class—the instance variables—from the design of the database.

For example, if your database stores the name of a user as a single `NAME` column, but your `User` class has `firstname` and `lastname` fields, you can add the following persistent `name` property to the class (this is example 2 from the `domainmodel` folder source code).

Listing 3.2 POJO implementation of the `User` class with logic in accessor methods

```
Path: Ch03/domainmodel/src/main/java/com/manning/javapersistence/ch03/e
➥ /User.java

public class User {

    private String firstname;
    private String lastname;

    public String getName() {
        return firstname + ' ' + lastname;
    }

    public void setName(String name) {
        StringTokenizer tokenizer = new StringTokenizer(name);
        firstname = tokenizer.nextToken();
        lastname = tokenizer.nextToken();
    }

}
```

Later you'll see that a custom type converter in the persistence service is a better way to handle many of these kinds of situations. It helps to have several options.

Another issue to consider is *dirty checking*. Hibernate automatically detects state changes so that it can synchronize the updated state with the database. It's usually safe to return a different instance from the getter method than the instance passed by Hibernate to the setter. Hibernate compares them by value—not by object identity—to determine whether the attribute's persistent state needs to be updated. For example, the following getter method doesn't result in unnecessary SQL `UPDATE` s:

```
public String getFirstname() {
    return new String(firstname);
}
```

There is an important point to note about *dirty checking* when persisting collections. If you have an `Item` entity with a `Set<Bid>` field that's accessed through the `setBids` setter, this code will result in an unnecessary SQL `UPDATE` :

```
item.setBids(bids);
em.persist(item);
item.setBids(bids);
```

This happens because Hibernate has its own collection implementations: `PersistentSet` , `PersistentList` , or `PersistentMap` . Providing setters for an entire collection is not good practice anyway.

How does Hibernate handle exceptions when your accessor methods throw them? If Hibernate uses accessor methods when loading and storing instances, and a `RuntimeException` (unchecked) is thrown, the current transaction is rolled back, and the exception is yours to handle in the code that called the Jakarta Persistence (or native Hibernate) API. If you throw a checked application exception, Hibernate wraps the exception into a `RuntimeException` .

Next we'll focus on the relationships between entities and the associations between persistent classes.

3.2.4 Implementing POJO associations

Let's now look at how you can associate and create different kinds of relationships between objects: one-to-many, many-to-one, and bidirectional relationships. We'll look at the scaffolding code needed to create these associations, how you can simplify relationship management, and how you can enforce the integrity of these relationships.

You can create properties to express associations between classes, and you will (typically) call accessor methods to navigate from instance to in-

stance at runtime. Let's consider the associations defined by the `Item` and `Bid` persistent classes, as shown in figure 3.4.

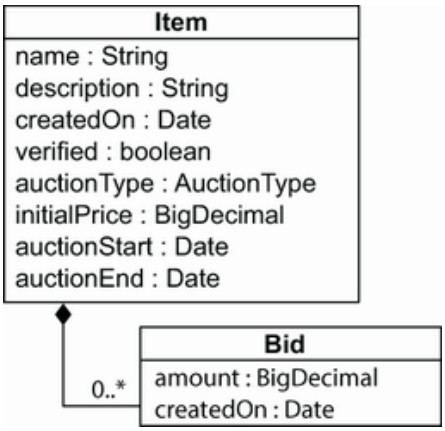

Figure 3.4 Associations between the `Item` and `Bid` classes

We left the association-related attributes, `Item#bids` and `Bid#item`, out of figure 3.4. These properties and the methods that manipulate their values are called *scaffolding code*. This is what the scaffolding code for the `Bid` class looks like:

```
Path: Ch03/domainmodel/src/main/java/com/manning/javapersistence/ch03/e
➥ /Bid.java

public class Bid {

    private Item item;

    public Item getItem() {
        return item;
    }

    public void setItem(Item item) {
        this.item = item;
    }

}
```

The `item` property allows navigation from a `Bid` to the related `Item`. This is an association with *many-to-one* multiplicity; users can make many bids for each item.

Here is the `Item` class's scaffolding code:

```
Path: Ch03/domainmodel/src/main/java/com/manning/javapersistence/ch03/e
➥ /Item.java

public class Item {
    private Set<Bid> bids = new HashSet<>();
```

```
        public Set<Bid> getBids() {
            return Collections.unmodifiableSet(bids);
        }
    }
```

This association between the two classes allows for *bidirectional* navigation: the *many-to-one* is from this perspective a *one-to-many* multiplicity. One item can have many bids—they are of the same type but were generated during the auction by different users and with different amounts, as shown in table 3.1.

Table 3.1 One `Item` has many `Bid`s generated during the auction

Item	Bid	User	Amount
1	1	John	100
1	2	Mike	120
1	3	John	140

The scaffolding code for the `bids` property uses a collection interface type, `java.util.Set`. JPA requires interfaces for collection-typed properties, where you must use `java.util.Set`, `java.util.List`, or `java.util.Collection` rather than `HashSet`, for example. It's good practice to program to collection interfaces anyway, rather than concrete implementations, so this restriction shouldn't bother you.

You can choose a `Set` and initialize the field to a new `HashSet`, because the application disallows duplicate bids. This is good practice, because you'll avoid any `NullPointerException`s when someone is accessing the property of a new `Item`, which will have an empty set of bids. The JPA provider is also required to set a non-empty value on any mapped collection-valued property, such as when an `Item` without bids is loaded from the database. (It doesn't have to use a `HashSet`; the implementation is up to the provider. Hibernate has its own collection implementations with additional capabilities, such as dirty checking.)

Shouldn't bids on an item be stored in a list?

The first reaction is often to preserve the order of elements as they're entered by users, because this may also be the order in which you will show them later. Certainly, in an auction application, there has to be a defined order in which the user sees bids for an item, such as the highest bid first or the newest bid last. You might even work with a `java.util.List` in your user interface code to sort and display bids for an item.

That doesn't mean this display order should be durable, however. The data integrity isn't affected by the order in which bids are displayed. You'll need to store the amount of each bid, so you can always find the highest bid, and you'll need to store a timestamp for when each bid is created, so you can always find the newest bid. When in doubt, keep your system flexible, and sort the data when it's retrieved from the data store (in a query) or shown to the user (in Java code), not when it's stored.

Accessor methods for associations need to be declared `public` only if they're part of the external interface of the persistent class used by the application logic to create a link between two instances. We'll now focus on this issue, because managing the link between an `Item` and a `Bid` is much more complicated in Java code than it is in an SQL database, with declarative foreign key constraints. In our experience, engineers are often unaware of this complication, which arises from a network object model with bidirectional references (pointers). Let's walk through the issue step by step.

The basic procedure for linking a `Bid` with an `Item` looks like this:

```
anItem.getBids().add(aBid);
aBid.setItem(anItem);
```

Whenever you create this bidirectional link, two actions are required:

- You must add the `Bid` to the `bids` collection of the `Item` (shown in figure 3.5).

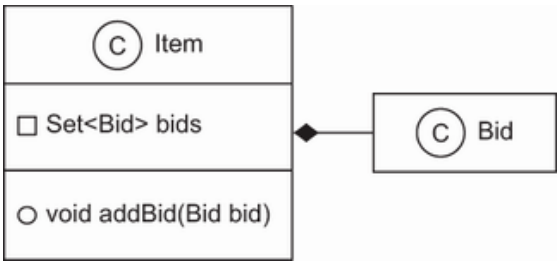

Figure 3.5 Step 1 of linking a `Bid` with an `Item`: adding a `Bid` to the set of `Bids` from the `Item`

- The `item` property of the `Bid` must be set (shown in figure 3.6).

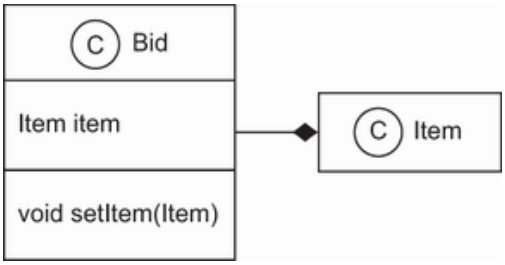

Figure 3.6 Step 2 of linking a `Bid` with an `Item`: setting the `Item` on the `Bid` side

JPA doesn't manage persistent associations. If you want to manipulate an association, you must write the same code you would write without Hibernate. If an association is bidirectional, you must consider both sides of the relationship. If you ever have problems understanding the behavior of associations in JPA, just ask yourself, "What would I do without Hibernate?" Hibernate doesn't change the regular Java semantics.

We recommend that you add convenience methods to group these operations, allowing reuse and helping ensure correctness, and in the end guaranteeing data integrity (a `Bid` *is required* to have a reference to an `Item`). The next listing shows such a convenience method in the `Item` class (this is example 3 from the `domainmodel` folder source code).

Listing 3.3 A convenience method simplifies relationship management

```
Path: Ch03/domainmodel/src/main/java/com/manning/javapersistence/ch03/e
➥ /Item.java

public void addBid(Bid bid) {
    if (bid == null)
        throw new NullPointerException("Can't add null Bid");
    if (bid.getItem() != null)
        throw new IllegalStateException(
                    "Bid is already assigned to an Item");
    bids.add(bid);
    bid.setItem(this);
}
```

The `addBid()` method not only reduces the lines of code when dealing with `Item` and `Bid` instances but also enforces the cardinality of the association. You avoid errors that arise from leaving out one of the two required actions. You should always provide this kind of grouping of operations for associations, if possible. If you compare this with the relational model of foreign keys in an SQL database, you can easily see how a network and a pointer model complicate a simple operation: instead of a declarative constraint, you need procedural code to guarantee data integrity.

Because you want `addBid()` to be the only externally visible mutator method for the bids of an item (possibly in addition to a `removeBid()` method), consider making the `Bid#setItem()` method package-visible.

The `Item#getBids()` getter method should not return a modifiable collection, so that clients can't use the collection to make changes that aren't reflected on the other side. Bids added directly to the collection may belong to an item, but they wouldn't have a reference to that item, which would create an inconsistent state, according to the database constraints. To prevent this problem, you can wrap the internal collection before returning it from the getter method with `Collections.unmodifiableCollection(c)` and `Collections.unmodifiableSet(s)`. The client will then get an exception if it tries to modify the collection You can therefore force every modification to go through the relationship management method, guaranteeing integrity. It is always good practice to return an unmodifiable collection from your classes so that the client does not have direct access to it.

An alternative strategy is to use immutable instances. For example, you could enforce integrity by requiring an `Item` argument in the constructor of `Bid`, as shown in the following listing (example 4 from the `domainmodel` folder source code).

Listing 3.4 Enforcing the integrity of relationships with a constructor

```
Path: Ch03/domainmodel/src/main/java/com/manning/javapersistence/ch03/e
➥ /Bid.java

public class Bid {

    private Item item;

    public Bid(Item item) {
        this.item = item;
        item.bids.add(this); // Bidirectional
    }

    public Item getItem() {
        return item;
    }
}
```

In this constructor, the `item` field is set; no further modification of the field value should occur. The collection on the other side is also updated for a bidirectional relationship, while the `bids` field from the `Item` class is now package-private. There is no `Bid#setItem()` method.

There are several problems with this approach, however. First, Hibernate can't call this constructor. You need to add a no-argument constructor for Hibernate, and it needs to be at least package-visible. Furthermore, because there is no `setItem()` method, Hibernate would have to be configured to access the `item` field directly. This means the field can't be `final`, so the class isn't guaranteed to be immutable.

It's up to you how many convenience methods and layers you want to wrap around the persistent association properties or fields, but we recommend being consistent and applying the same strategy to all your domain model classes. For the sake of readability, we won't always show our convenience methods, special constructors, and other such scaffolding in future code samples; you should add them according to your own taste and requirements.

You now have seen domain model classes and how to represent their attributes and the relationships between them. Next we'll increase the level of abstraction: we'll add metadata to the domain model implementation and declare aspects such as validation and persistence rules.

3.3 Domain model metadata

Metadata is data about data, so domain model metadata is information about your domain model. For example, when you use the Java Reflection API to discover the names of classes in your domain model or the names of their attributes, you're accessing domain model metadata.

ORM tools also require metadata to specify the mapping between classes and tables, properties and columns, associations and foreign keys, Java types and SQL types, and so on. This object/relational mapping metadata governs the transformation between the different type systems and relationship representations in object-oriented and SQL systems. JPA has a metadata API that you can call to obtain details about the persistence aspects of your domain model, such as the names of persistent entities and attributes. It's your job as an engineer to create and maintain this information.

JPA standardizes two metadata options: annotations in Java code and externalized XML descriptor files. Hibernate has some extensions for native functionality, also available as annotations or XML descriptors. We usually prefer annotations as the primary source of mapping metadata. After reading this section, you'll have the information to make an informed decision for your own project.

We'll also discuss *Bean Validation* (JSR 303) in this section, and how it provides declarative validation for your domain model (or any other) classes. The reference implementation of this specification is the *Hibernate*

Validator project. Most engineers today prefer Java annotations as the primary mechanism for declaring metadata.

3.3.1 Annotation-based metadata

The big advantage of annotations is that they put metadata, such as `@Entity`, next to the information it describes, instead of separating it in a different file. Here's an example:

```
import javax.persistence.Entity;
@Entity
public class Item {
}
```

You can find the standard JPA mapping annotations in the `javax.persistence` package. This example declares the `Item` class as a persistent entity using the `@javax .persistence.Entity` annotation. All of its attributes are now automatically made persistent with a default strategy. That means you can load and store instances of `Item`, and all the properties of the class are part of the managed state.

Annotations are type-safe, and the JPA metadata is included in the compiled class files. The annotations are still accessible at runtime, and Hibernate reads the classes and metadata with Java reflection when the application starts. The IDE can also easily validate and highlight annotations—they're regular Java types, after all. When you refactor your code, you rename, delete, and move classes and properties. Most development tools and editors can't refactor XML elements and attribute values, but annotations are part of the Java language and are included in all refactoring operations.

Is my class now dependent on JPA?

You need JPA libraries on your classpath when you compile the source of your domain model class. JPA isn't required on the classpath when you create an instance of the class, such as in a client application that doesn't execute any JPA code. Only when you access the annotations through reflection at runtime (as Hibernate does internally when it reads your metadata) will you need the packages on the classpath.

When the standardized Jakarta Persistence annotations are insufficient, a JPA provider may offer additional annotations.

Using vendor extensions

Even if you map most of your application's model with JPA-compatible annotations from the `javax.persistence` package, you may have to

use vendor extensions at some point. For example, some performance-tuning options you'd expect to be available in high-quality persistence software are only available as Hibernate-specific annotations. This is how JPA providers compete, so you can't avoid annotations from other packages—there's a reason why you chose to use Hibernate.

The following snippet shows the `Item` entity source code again with a Hibernate-only mapping option:

```
import javax.persistence.Entity;
@Entity
@org.hibernate.annotations.Cache(
    usage = org.hibernate.annotations.CacheConcurrencyStrategy.READ_WRI
)
public class Item {
}
```

We prefer to prefix Hibernate annotations with the full `org.hibernate.annotations` package name. Consider this good practice, because it enables you to easily see which metadata for this class is from the JPA specification and which is vendor-specific. You can also easily search your source code for `org.hibernate.annotations` and get a complete overview of all nonstandard annotations in your application in a single search result.

If you switch your Jakarta Persistence provider, you'll only have to replace the vendor-specific extensions, and you can expect a similar feature set to be available from most mature JPA implementations. Of course, we hope you'll never have to do this, and it doesn't often happen in practice—just be prepared.

Annotations on classes only cover metadata that applies to that particular class. You'll often also need metadata at a higher level for an entire package or even the whole application.

Global annotation metadata

The `@Entity` annotation maps a particular class. JPA and Hibernate also have annotations for global metadata. For example, a `@NamedQuery` has a global scope; you don't apply it to a particular class. Where should you place this annotation?

Although it's possible to place such global annotations in the source file of a class (at the top of any class), we prefer to keep global metadata in a separate file. Package-level annotations are a good choice; they're in a file called package-info.java in a particular package directory. You will be able to look for them in a single place instead of browsing through sever-

al files. The following listing shows an example of global named query declarations (example 5 from the `domainmodel` folder source code).

Listing 3.5 Global metadata in a package-info.java file

```
Path: Ch03/domainmodel/src/main/java/com/manning/javapersistence/ch03/e
➥ /package-info.java

@org.hibernate.annotations.NamedQueries({
    @org.hibernate.annotations.NamedQuery(
        name = "findItemsOrderByName",
        query = "select i from Item i order by i.name asc"
    )
    ,
    @org.hibernate.annotations.NamedQuery(
        name = "findItemBuyNowPriceGreaterThan",
        query = "select i from Item i where i.buyNowPrice > :price",
        timeout = 60, // Seconds!
        comment = "Custom SQL comment"
    )
})

package com.manning.javapersistence.ch03.ex05;
```

Unless you've used package-level annotations before, the syntax of this file with the package and import declarations at the bottom is probably new to you.

Annotations will be our primary tool for ORM metadata throughout this book, and there is much to learn about this subject. Before we look at the alternative mapping style using XML files, let's use some simple annotations to improve our domain model classes with validation rules.

3.3.2 Applying constraints to Java objects

Most applications contain a multitude of data integrity checks. When you violate one of the simplest data-integrity constraints, you may get a `NullPointerException` because a value isn't available. You may similarly get this exception when a string- valued property shouldn't be empty (an empty string isn't `null`), when a string has to match a particular regular expression pattern, or when a number or date value must be within a certain range.

These business rules affect every layer of an application: The user interface code has to display detailed and localized error messages. The business and persistence layers must check input values received from the client before passing them to the data store. The SQL database must be the final validator, guaranteeing the integrity of durable data.

The idea behind Bean Validation is that declaring rules such as “This property can’t be null” or “This number has to be in the given range” is much easier and less error-prone than repeatedly writing if-then-else procedures. Furthermore, declaring these rules on the central component of your application, the domain model implementation, enables integrity checks in every layer of the system. The rules are then available to the presentation and persistence layers. And if you consider how data-integrity constraints affect not only your Java application code but also your SQL database schema—which is a collection of integrity rules—you might think of Bean Validation constraints as additional ORM metadata.

Look at the following extended `Item` domain model class from the `validation` folder source code.

Listing 3.6 Applying validation constraints on `Item` entity fields

```
Path: Ch03/validation/src/main/java/com/manning/javapersistence/ch03
➥ /validation/Item.java

import javax.validation.constraints.Future;
import javax.validation.constraints.NotNull;
import javax.validation.constraints.Size;
import java.util.Date;

public class Item {
    @NotNull
    @Size(
            min = 2,
            max = 255,
            message = "Name is required, maximum 255 characters."
    )
    private String name;

    @Future
    private Date auctionEnd;
}
```

We add two attributes when an auction concludes: the `name` of an item and the `auctionEnd` date. Both are typical candidates for additional constraints. First, we want to guarantee that the name is always present and human-readable (one-character item names don’t make much sense) but not too long—your SQL database will be most efficient with variable-length strings up to 255 characters, and your user interface will also have some constraints on visible label space. Second, the ending time of an auction should obviously be in the future. If we don’t provide an error message for a constraint, a default message will be used. Messages can be keys to external property files for internationalization.

The validation engine will access the fields directly if you annotate the fields. If you prefer to use calls through accessor methods, annotate the getter method with validation constraints, not the setter (annotations on setters are not supported). Then the constraints will be part of the class's API and will be included in its Javadoc, making the domain model implementation easier to understand. Note that constraints being part of the class's API is independent of access by the JPA provider; for example, Hibernate Validator may call accessor methods, whereas Hibernate ORM may call fields directly.

Bean Validation isn't limited to built-in annotations; you can create your own constraints and annotations. With a custom constraint, you can even use class-level annotations and validate several attribute values at the same time on an instance of a class. The following test code shows how you could manually check the integrity of an `Item` instance.

Listing 3.7 Testing an `Item` instance for constraint violations

```
Path: Ch03/validation/src/test/java/com/manning/javapersistence/ch03
➥ /validation/ModelValidation.java

ValidatorFactory factory = Validation.buildDefaultValidatorFactory();
Validator validator = factory.getValidator();

Item item = new Item();
item.setName("Some Item");
item.setAuctionEnd(new Date());

Set<ConstraintViolation<Item>> violations = validator.validate(item);

ConstraintViolation<Item> violation = violations.iterator().next();
String failedPropertyName =
        violation.getPropertyPath().iterator().next().getName();

// Validation error, auction end date was not in the future!
assertAll(() -> assertEquals(1, violations.size()),
        () -> assertEquals("auctionEnd", failedPropertyName),
        () -> {
            if (Locale.getDefault().getLanguage().equals("en"))
                assertEquals(violation.getMessage(),
                             "must be a future date");
        });
```

We're not going to explain this code in detail but offer it for you to explore. You'll rarely write this kind of validation code; usually this validation is automatically handled by your user interface and persistence framework. It's therefore important to look for Bean Validation integration when selecting a UI framework.

Hibernate, as required from any JPA provider, also automatically integrates with Hibernate Validator if the libraries are available on the classpath, and it offers the following features:

- You don't have to manually validate instances before passing them to Hibernate for storage.
- Hibernate recognizes constraints on persistent domain model classes and triggers validation before database insert or update operations. When validation fails, Hibernate throws a `ConstraintViolationException` containing the failure details to the code calling persistence-management operations.
- The Hibernate toolset for automatic SQL schema generation understands many constraints and generates SQL DDL-equivalent constraints for you. For example, a `@NotNull` annotation translates into an SQL `NOT NULL` constraint, and a `@Size(n)` rule defines the number of characters in a `VARCHAR(n)` -typed column.

You can control this behavior of Hibernate with the `<validation-mode>` element in your persistence.xml configuration file. The default mode is `AUTO` , so Hibernate will only validate if it finds a Bean Validation provider (such as Hibernate Validator) on the classpath of the running application. With the `CALLBACK` mode, validation will always occur, and you'll get a deployment error if you forget to bundle a Bean Validation provider. The `NONE` mode disables automatic validation by the JPA provider.

You'll see Bean Validation annotations again later in this book; you'll also find them in the example code bundles. We could write much more about Hibernate Validator, but we'd only repeat what is already available in the project's excellent reference guide (**http://mng.bz/ne65)**. Take a look and find out more about features such as validation groups and the metadata API for the discovery of constraints.

3.3.3 Externalizing metadata with XML files

You can replace or override every annotation in JPA with an XML descriptor element. In other words, you don't have to use annotations if you don't want to, or if keeping mapping metadata separate from source code is advantageous to your system design for whatever reason. Keeping the mapping metadata separate has the benefit of not cluttering the JPA annotations with Java code and of making your Java classes more reusable, though you lose the type-safety. This approach is less in use nowadays, but we'll analyze it anyway, as you may still encounter it or choose this approach for your own projects.

The following listing shows a JPA XML descriptor for a particular persistence unit (the `metadataxmljpa` folder source code).

Listing 3.8 JPA XML descriptor containing the mapping metadata of a persistence unit

```
Path: Ch03/metadataxmljpa/src/test/resources/META-INF/orm.xml

<entity-mappings
        version="2.2"
        xmlns="http://xmlns.jcp.org/xml/ns/persistence/orm"
        xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/persistence/orm
            http://xmlns.jcp.org/xml/ns/persistence/orm_2_2.xsd">

 <persistence-unit-metadata>
        <xml-mapping-metadata-complete/>
        <persistence-unit-defaults>
            <delimited-identifiers/>
        </persistence-unit-defaults>
 </persistence-unit-metadata>
 <entity class="com.manning.javapersistence.ch03.metadataxmljpa.Item"
                access="FIELD">
    <attributes>
        <id name="id">
            <generated-value strategy="AUTO"/>
        </id>
        <basic name="name"/>
        <basic name="auctionEnd">
            <temporal>TIMESTAMP</temporal>
        </basic>
    </attributes>
 </entity>
</entity-mappings>
```

Ⓐ Declare the global metadata.

Ⓑ Ignore all mapping annotations. If we include the `<xml-mapping-metadata- complete>` element, the JPA provider ignores all annotations on the domain model classes in this persistence unit and relies only on the mappings as defined in the XML descriptors.

Ⓒ The default settings escape all SQL columns, tables, and other names.

Ⓓ Escaping is useful if the SQL names are actually keywords (a "USER" table, for example).

Ⓔ Declare the `Item` class as an entity with field access.

Ⓕ Its attributes are the `id`, which is autogenerated, the `name`, and the `auctionEnd`, which is a temporal field.

The JPA provider automatically picks up this descriptor if you place it in a META-INF/ orm.xml file on the classpath of the persistence unit. If you prefer to use a different filename or several files, you'll have to change the configuration of the persistence unit in your META-INF/persistence.xml file:

```
<persistence-unit name="persistenceUnitName">
     . . .
    <mapping-file>file1.xml</mapping-file>
    <mapping-file>file2.xml</mapping-file>
     . . .
</persistence-unit>
```

If you don't want to ignore the annotation metadata but override it instead, don't mark the XML descriptors as "complete," and name the class and property you want to override:

```
<entity class="com.manning.javapersistence.ch03.metadataxmljpa.Item">
    <attributes>
         <basic name="name">
            <column name="ITEM_NAME"/>
        </basic>
    </attributes>
</entity>
```

Here we map the `name` property to the `ITEM_NAME` column; by default, the property would map to the `NAME` column. Hibernate will now ignore any existing annotations from the `javax.persistence.annotation` and `org.hibernate.annotations` packages on the `name` property of the `Item` class. But Hibernate won't ignore Bean Validation annotations and still applies them for automatic validation and schema generation! All other annotations on the `Item` class are also recognized. Note that we don't specify an access strategy in this mapping, so field access or accessor methods are used, depending on the position of the `@Id` annotation in `Item`. (We'll get back to this detail in the next chapter.)

We won't talk much about JPA XML descriptors in this book. The syntax of these documents is the same as the JPA annotation syntax, so you shouldn't have any problems writing them. We'll focus on the important aspect: the mapping strategies.

3.3.4 Accessing metadata at runtime

The JPA specification provides programming interfaces for accessing the metamodel (information about the model) of persistent classes. There are two flavors of the API. One is more dynamic in nature and similar to basic Java reflection. The second option is a static metamodel. For both options, access is read-only; you can't modify the metadata at runtime.

The dynamic Metamodel API in Jakarta Persistence

Sometimes you'll want to get programmatic access to the persistent attributes of an entity, such as when you want to write custom validation or generic UI code. You'd like to know dynamically what persistent classes and attributes your domain model has. The code in the following listing shows how you can read metadata with Jakarta Persistence interfaces (from the `metamodel` folder source code).

Listing 3.9 Obtaining entity type information with the Metamodel API

```
Path: Ch03/metamodel/src/test/java/com/manning/javapersistence/ch03
➥ /metamodel/MetamodelTest.java

Metamodel metamodel = emf.getMetamodel();
Set<ManagedType<?>> managedTypes = metamodel.getManagedTypes();
ManagedType<?> itemType = managedTypes.iterator().next();

assertAll(() -> assertEquals(1, managedTypes.size()),
        () -> assertEquals(
                Type.PersistenceType.ENTITY,
                itemType.getPersistenceType()));
```

You can get the `Metamodel` object from either the `EntityManagerFactory`, of which you'll typically have only one instance per data source in an application, or, if it's more convenient, by calling `EntityManager#getMetamodel()`. The set of managed types contains information about all persistent entities and embedded classes (which we'll discuss in the next chapter). In this example, there's only one managed type: the `Item` entity. This is how you can dig deeper and find out more about each attribute.

Listing 3.10 Obtaining entity attribute information with the Metamodel API

```
Path: Ch03/metamodel/src/test/java/com/manning/javapersistence/ch03
➥ /metamodel/MetamodelTest.java

SingularAttribute<?, ?> idAttribute =
        itemType.getSingularAttribute("id");
```

```
assertFalse(idAttribute.isOptional());

SingularAttribute<?, ?> nameAttribute =
        itemType.getSingularAttribute("name");

assertAll(() -> assertEquals(String.class, nameAttribute.getJavaType()),
        () -> assertEquals(
                Attribute.PersistentAttributeType.BASIC,
                nameAttribute.getPersistentAttributeType()
        ));

SingularAttribute<?, ?> auctionEndAttribute =
        itemType.getSingularAttribute("auctionEnd");
assertAll(() -> assertEquals(Date.class,
                             auctionEndAttribute.getJavaType()),
          () -> assertFalse(auctionEndAttribute.isCollection()),
          () -> assertFalse(auctionEndAttribute.isAssociation())
);
```

Ⓐ The attributes of the entity are accessed with a string: the `id`.

Ⓑ Check that the `id` attribute is not optional. This means that it cannot be NULL, since it is the primary key.

ⒸThe `name`.

ⒹCheck that the `name` attribute has the `String` Java type and the basic persistent attribute type.

Ⓔ The `auctionEnd` date. This obviously isn't type-safe, and if you change the names of the attributes, this code will be broken and obsolete. The strings aren't automatically included in the refactoring operations of your IDE.

Ⓕ Check that the `auctionEnd` attribute has the `Date` Java type and that it is not a collection or an association.

JPA also offers a static type-safe metamodel.

Using a static metamodel

In Java (at least up to version 17), you can't access the fields or accessor methods of a bean in a type-safe fashion—only by their names, using strings. This is particularly inconvenient with JPA criteria querying, which is a type-safe alternative to string-based query languages. Here's an example:

```
Path: Ch03/metamodel/src/test/java/com/manning/javapersistence/ch03
➥ /metamodel/MetamodelTest.java

CriteriaBuilder cb = em.getCriteriaBuilder();
CriteriaQuery<Item> query = cb.createQuery(Item.class);      Ⓐ
Root<Item> fromItem = query.from(Item.class);
query.select(fromItem);
List<Item> items = em.createQuery(query).getResultList();

assertEquals(2, items.size());
```

Ⓐ The query is the equivalent of `select i from Item i`. This query returns all items in the database, and there are two in this case. If you want to restrict this result and only return items with a particular name, you will have to use a `like` expression, comparing the `name` attribute of each item with the pattern set in a parameter.

The following code introduces a filter on the read operation:

```
Path: Ch03/metamodel/src/test/java/com/manning/javapersistence/ch03
➥ /metamodel/MetamodelTest.java

Path<String> namePath = fromItem.get("name");
query.where(cb.like(namePath, cb.parameter(String.class, "pattern")));
List<Item> items = em.createQuery(query).
                    setParameter("pattern", "%Item 1%").
                    getResultList();
assertAll(() -> assertEquals(1, items.size()),
        () -> assertEquals("Item 1", items.iterator().next().getName())
```

Ⓐ The query is the equivalent of `select i from Item i where i.name like :pattern`. Notice that the `namePath` lookup requires the `name` string. This is where the type-safety of the criteria query breaks down. You can rename the `Item` entity class with your IDE's refactoring tools, and the query will still work. But as soon as you touch the `Item#name` property, manual adjustments are necessary. Luckily, you'll catch this when the test fails.

A much better approach that's safe for refactoring and that detects mismatches at compile time and not runtime is the type-safe static metamodel:

```
Path: Ch03/metamodel/src/test/java/com/manning/javapersistence/ch03
➥ /metamodel/MetamodelTest.java

query.where(
    cb.like(
        fromItem.get(Item_.name),
```

```
            cb.parameter(String.class, "pattern")
        )
    );
```

The special class here is `Item_`; note the underscore. This class is a metadata class, and it lists all the attributes of the `Item` entity class:

```
Path: Ch03/metamodel/target/classes/com/manning/javapersistence/ch03
➥ /metamodel/Item_.class

@Generated(value = "org.hibernate.jpamodelgen.JPAMetaModelEntityProcess
@StaticMetamodel(Item.class)
public abstract class Item_ {

   public static volatile SingularAttribute<Item, Date> auctionEnd;
   public static volatile SingularAttribute<Item, String> name;
   public static volatile SingularAttribute<Item, Long> id;

   public static final String AUCTION_END = "auctionEnd";
   public static final String NAME = "name";
   public static final String ID = "id";

}
```

This class will be automatically generated. The Hibernate JPA 2 Metamodel Generator (a subproject of the Hibernate suite) takes care of this. Its only purpose is to generate static metamodel classes from your managed persistent classes. You need to add this Maven dependency to your pom.xml file:

```
Path: Ch03/metamodel/pom.xml

<dependency>
    <groupId>org.hibernate</groupId>
    <artifactId>hibernate-jpamodelgen</artifactId>
    <version>5.6.9.Final</version>
</dependency>
```

It will run automatically whenever you build your project and will generate the appropriate `Item_` metadata class. You will find the generated classes in the target\ generated-sources folder.

This chapter discussed the construction of the domain model and the dynamic and static metamodel. Although you've seen some mapping constructs in the previous sections, we haven't introduced any more sophisticated class and property mappings so far. You should now decide which mapping metadata strategy you'd like to use in your project—we recom-

mend the more commonly used annotations, instead of the already less used XML. Then you can read more about class and property mappings in part 2 of this book, starting with chapter 5.

Summary

- We analyzed different abstract notions like the information model and data model, and then jumped into JPA/Hibernate so that we could work with databases from Java programs.
- You can implement persistent classes free of any cross-cutting concerns like logging, authorization, and transaction demarcation.
- The persistent classes only depend on JPA at compile time.
- Persistence-related concerns should not leak into the domain model implementation.
- Transparent persistence is important if you want to execute and test the business objects independently.
- The POJO concept and the JPA entity programming model have several things in common, arising from the old JavaBean specification: they implement properties as private or protected member fields, while property accessor methods are generally public or protected.
- We can access the metadata using either the dynamic metamodel or the static metamodel.